

Java Take-Home Assignment

Mid-Level Backend Engineer

Overview

Build a small Spring Boot service that exposes a foreign-exchange API: fetches live rates from a public provider, performs currency conversions against per-client account balances, and stores a history of conversions for later lookup. The goal is to give us a clear, honest view of how you write production-grade backend code.

Time and submission

- **Target time:** approximately 7 hours.
- If you find yourself well past that, stop and write down what you would do next in the README — that is a deliverable too.
- Submit a Git repository link (GitHub, GitLab, or Bitbucket) with a README.md describing how to run it, your trade-offs, and anything you would have done with more time.

What you will build

A Spring Boot service with the following REST endpoints:

- **GET /rates?from=USD&to=EUR** → the current exchange rate between two currencies.
- **POST /conversions** → convert an amount for a given client. The request must carry a client identifier (see Per-client balances below). Returns transactionId, sourceAmount, sourceCurrency, targetAmount, targetCurrency, rate, timestamp, and the client's updated balances.
- **GET /conversions?transactionId=...&date=YYYY-MM-DD&clientId=...&page=...&size=...** → paginated history filtered by transactionId, date, or clientId (at least one filter must be provided).
- **GET /clients/{clientId}/balances** → the client's current balances, one row per currency held.

Per-client account balances

Conversions run against per-client balances held in your database. There is no authentication in this assignment — the client identifier is passed by the caller, e.g. via an X-Client-Id header on POST /conversions (you may use the request body instead if you prefer; document your choice).

- **Seed initial balances** for at least two demo clients via a SQL migration (e.g. CLIENT-001 with 10,000 USD and 8,000 EUR; CLIENT-002 with 5,000 GBP). Document the demo clients in the README so we can run conversions out of the box.

- **On a successful conversion:** debit the source currency balance and credit the target currency balance atomically, and persist the conversion record. Both sides must land in the same database transaction.
- **Insufficient funds:** if the source balance is below the source amount, return HTTP 422 with an `INSUFFICIENT_FUNDS` error code. Do not persist a conversion record in this case.
- **Unknown client or unknown currency for the client:** return HTTP 404 with a clear error code (e.g. `CLIENT_NOT_FOUND`, `BALANCE_NOT_FOUND`).
- **Concurrency:** two simultaneous `POST /conversions` requests for the same client must not double-spend. Pick a strategy (pessimistic row lock, optimistic locking with `@Version`, or a serialised single-writer per client) and explain it in the README.
- You do NOT need to build deposit/withdraw endpoints. Adding one is fine if it helps you test, but seeding via Flyway is sufficient.

Technical requirements

- Spring Boot 3.x on Java 17 or newer.
- **Money as BigDecimal** — explicit scale and rounding mode. Do not use double or float for amounts.
- Persistence via JPA. H2 in-memory is acceptable; PostgreSQL with Testcontainers for the integration test is a strong plus.
- Schema migrations with Flyway (or Liquibase). Do not rely on `ddl-auto=update` in the production profile.
- **Idempotency on POST /conversions.** The client may pass an Idempotency-Key header. Replays of the same key within the persistence window must return the original result, not create a duplicate and not double-debit the balance. Briefly explain your approach in the README.
- **External provider integration** with timeouts and graceful failure — when the provider is down the request must fail with a meaningful error, not a stack trace.
- **Caching** of rates with a TTL. Explain your invalidation choice in the README.
- **Validation** — ISO-4217 currency codes, positive amounts, sane bounds, non-blank client id.
- **Error handling** — a `@ControllerAdvice` returning consistent error bodies with codes, not raw stack traces. Use distinct codes for `INSUFFICIENT_FUNDS`, `CLIENT_NOT_FOUND`, `BALANCE_NOT_FOUND`, and provider failures.
- **OpenAPI / Swagger UI** auto-generated (e.g. SpringDoc).
- **Tests** — at least one integration test that exercises the controller through the Spring context, plus unit tests for the conversion logic. Test idempotency replay, insufficient funds, and the happy-path debit/credit explicitly.
- **Self-contained** — ``docker compose up`` (or ``./mvnw spring-boot:run``) must boot the service with no manual setup beyond environment variables documented in the README.
- **Dockerfile** — multi-stage build, non-root user.

Suggested FX rate providers (free, no card required)

- <https://open.er-api.com/v6/latest/USD>
- <https://api.frankfurter.dev/v1/latest?from=USD>
- <https://api.exchangerate.host/>

You may use any provider. If your chosen one needs a key, document how to set it.

AI tooling

You may use AI coding assistants (Claude Code, Cursor, Copilot, etc.). We expect modern tooling. In the live follow-up interview you will extend this code yourself, so make sure you understand every line you submit.

Non-goals — please do NOT spend time on these

- Authentication / OAuth — the clientId is passed by the caller, full stop.
- UI or frontend.
- Kubernetes manifests, Helm, CI/CD pipelines.
- Microservice splitting, event sourcing, CQRS.
- A full ledger / journal-entry model. A single balance row per client × currency is enough.
- Inventing or stacking design patterns. Use the simplest structure that does the job clearly.

How we will grade

Five axes, equally weighted. We share this rubric so you know exactly what we are looking for:

- **Correctness** — endpoints behave per spec; money math is precise; idempotency works; balances debit/credit atomically; insufficient funds is rejected.
- **Code clarity** — good names, separation of concerns, no premature abstractions.
- **Testing discipline** — meaningful tests rather than coverage theatre; at least one integration test; idempotency and insufficient-funds covered.
- **Production sense** — error handling, validation, timeouts, caching, schema migrations, concurrency strategy, container hygiene.
- **Communication** — README explains the why, lists trade-offs (especially around the concurrency choice), names what you would do next with more time.

Next step after submission — live follow-up

If your submission moves forward, we will schedule a 60–90 minute live session. You will share your screen and extend your own take-home with a small feature we describe at the start of the call. The goal is not to trick you — it is to see how you reason about a realistic small change in code you wrote.

Good luck, and we look forward to reading your code.